

Algorytmy Zachłanne – Wykład

22 kwietnia 2026

1 Matroidy

Definicja 1. Matroid to para $M = (S, \mathcal{A})$ taka, że:

1. S – niepusty zbiór skończony,
2. $\mathcal{A} \subseteq 2^S$ i $\mathcal{A} \neq \emptyset$ – czyli $\mathcal{A}$ jest niepustą rodziną podzbiorów S ,
3. $\forall A, B \subseteq S \quad A \in \mathcal{A} \wedge B \subseteq A \Rightarrow B \in \mathcal{A}$,
4. $\forall A, B \subseteq S \quad A, B \in \mathcal{A} \wedge |B| > |A| \Rightarrow \exists x \in B \setminus A$ takie, że $A \cup \{x\} \in \mathcal{A}$ (własność wymiany).

$\mathcal{A}$ – rodzina zbiorów niezależnych matroidu M . Elementy $\mathcal{A}$ – zbiory niezależne matroidu M .

1.1 Przykłady

Przykład 1. A – macierz, S – zbiór wierszy A ,

$$\mathcal{A} = \{R \subseteq S : R \text{ tworzy zbiór liniowo niezależny}\}.$$

$(S, \mathcal{A})$ – matroid.

Przykład 2. $G = (V, E)$ – graf,

$$\mathcal{A} = \{F \subseteq E : G[F] \text{ jest acykliczny}\}.$$

$(E, \mathcal{A})$ – matroid zwany **matroidem grafowym**.

Z definicji $\emptyset \in \mathcal{A}$.

1.2 Własności

Stwierdzenie 1. $M = (S, \mathcal{A})$. Wszystkie maksymalne zbiory niezależne M mają tę samą liczbę.

Dowód. Niech $A, B \in \mathcal{A}$ będą maksymalnymi zbiorami niezależnymi M . Przypuśćmy, że $|B| > |A|$. Z własności 4 istnieje $x \in B \setminus A$ takie, że $A \cup \{x\} \in \mathcal{A}$ – sprzeczność z maksymalnością A . $\square$

2 Problem optymalizacji na matroidzie

$M = (S, \mathcal{A})$ – matroid, $w: S \rightarrow \mathbb{R}_+$ – funkcja wagi.

Dla $A \subseteq S$ niech

$$w(A) = \sum_{x \in A} w(x) \rightarrow \text{waga } A.$$

Problem: Znaleźć zbiór niezależny o maksymalnej wadze w M .

Niech $S = \{x_1, x_2, \dots, x_n\}$.

2.1 Algorytm Greedy

Algorithm 1 Greedy(M, w)

```

1:  $n \leftarrow |S|$ 
2:  $A \leftarrow \emptyset$   $\{A - \text{rozwiązanie}\}$ 
3: posortuj  $S$  w porządku nierosnącym względem wagi, tak żeby  $w(x_1) \geq w(x_2) \geq \dots \geq w(x_n)$ 
4: for  $i \leftarrow 1$  to  $n$  do
5:   if  $A \cup \{x_i\} \in \mathcal{A}$  then
6:      $A \leftarrow A \cup \{x_i\}$ 
7:   end if
8: end for
9: return  $A$ 

```

2.2 Poprawność algorytmu

Twierdzenie 1 (Rado, Edmonds). Algorytm Greedy znajduje zbiór niezależny o maksymalnej wadze.

Dowód. Niech $A = \{x_{i_1}, x_{i_2}, \dots, x_{i_k}\}$ będzie zbiorem zwróconym przez algorytm i $w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_k})$.

Krok 1. Najpierw pokażemy, że A jest maksymalnym zbiorem niezależnym.

Przypuśćmy, że istnieje $x_j \in S \setminus A$ takie, że $A \cup \{x_j\} \in \mathcal{A}$. Wtedy $j < i_k$, bo w przeciwnym przypadku algorytm dodałby x_j do A jako $(k+1)$ -szy element.

Niech ℓ będzie najmniejszą liczbą taką, że $j < i_\ell$. Wtedy

$$w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_{\ell-1}}) \geq w(x_j) \geq w(x_{i_\ell}).$$

Ale wtedy algorytm dodałby x_j zamiast x_{i_ℓ} jako ℓ -ty element dodany do A – sprzeczność.

Zatem A jest maksymalnym zbiorem niezależnym.

Krok 2. Rozważmy zbiór niezależny $B = \{y_1, y_2, \dots, y_m\} \in \mathcal{A}$ i załóżmy, że $w(y_1) \geq w(y_2) \geq \dots \geq w(y_m)$.

Mamy $m \leq k$, bo A jest maksymalnym zbiorem niezależnym (Stwierdzenie 1).

Pokażemy, że $w(B) \leq w(A)$.

Pokażemy, że dla każdego $j \leq m$: $w(y_j) \leq w(x_{i_j})$.

Przypuśćmy, że $w(y_j) > w(x_{i_j})$. Rozważmy zbiory niezależne

$$A_{j-1} = \{x_{i_1}, \dots, x_{i_{j-1}}\} \subseteq A, \quad B_j = \{y_1, \dots, y_j\} \subseteq B.$$

$j = |B_j| > |A_{j-1}| = j - 1$, więc z własności 4 istnieje $y_s \in B_j \setminus A_{j-1}$ takie, że $A_{j-1} \cup \{y_s\} \in \mathcal{A}$.

Mamy $w(y_s) \geq w(y_j) > w(x_{i_j})$, i więc istnieje $t \leq j$ takie, że

$$w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_{t-1}}) \geq w(y_s) > w(x_{i_t}).$$

Z własności 3: $\{x_{i_1}, x_{i_2}, \dots, x_{i_{t-1}}, y_s\} \subseteq A_{j-1} \cup \{y_s\}$ jest zbiorem niezależnym.

Ale wtedy algorytm dodałby y_s zamiast x_{i_t} jako t -ty element dodany do A – sprzeczność.

Zatem

$$w(y_1) \leq w(x_{i_1}), \quad w(y_2) \leq w(x_{i_2}), \quad \dots, \quad w(y_m) \leq w(x_{i_m}),$$

a więc

$$w(B) = w(y_1) + \dots + w(y_m) \leq w(x_{i_1}) + \dots + w(x_{i_m}) \leq w(A). \quad \square$$

2.3 Złożoność czasowa algorytmu Greedy

Niech $n = |S|$.

linia 1	$O(n)$
linia 2	$O(1)$
linia 3	$O(n \log n)$
liczba iteracji pętli w linii 4	wynosi n
linia 5	$f(n)$ – czas sprawdzenia warunku z linii 5
linia 6	$O(1)$
linia 7	$O(1)$

Całkowita złożoność wynosi $O(n \log n + n \cdot f(n))$.

3 Problem szeregowania zadań

$S = \{z_1, z_2, \dots, z_n\}$ – zbiór zadań o jednostkowym czasie wykonania.

$d_1, d_2, \dots, d_n$ – deadlines, $d_i \in \{1, \dots, n\}$ dla $i \in [n]$.

$w_1, w_2, \dots, w_n$ – kary, $w_i \geq 0$ dla $i \in [n]$.

Płacimy karę w_i za zadanie z_i jeśli nie jest skończone w momencie d_i .

Problem: Znaleźć kolejność wykonania zadań minimalizującą sumę kar.

Założmy, że mamy pewien harmonogram. Zadanie z_i nazywamy **terminowym** w tym harmonogramie, jeśli jest zakończone do chwili d_i . W przeciwnym razie nazywamy je **spóźnionym**.

Nie płacimy kar za zadania terminowe, płacimy kary za zadania spóźnione.

Lemat 1. Każdy harmonogram można przetransformować na harmonogram z taką samą sumą kar, w którym wszystkie zadania terminowe poprzedzają wszystkie zadania spóźnione oraz zadania terminowe są posortowane w kolejności niemalejącej względem deadline'ów.

Dowód. **1.** Niech z_i będzie zadaniem spóźnionym ukończonym w chwili $l_i > d_i$ i niech z_j będzie zadaniem terminowym ukończonym w chwili $l_j \leq d_j$, i niech z_i będzie przed z_j w harmonogramie, tzn. $l_i < l_j$.

Zamieniamy miejscami z_i i z_j . Po zamianie:

- z_i jest ukończone w chwili $l_j > l_i \geq d_i$, i więc nadal jest spóźnione.
- z_j jest ukończone w chwili $l_i < l_j \leq d_j$, więc nadal jest terminowe.

Po pewnej liczbie takich zamian otrzymujemy harmonogram o tej samej sumie kar, w którym wszystkie zadania terminowe poprzedzają wszystkie zadania spóźnione.

2. Niech z_i, z_j będą zadaniami terminowymi takimi, że z_i jest ukończone w chwili $l_i \leq d_i$, z_j jest ukończone w chwili $l_j \leq d_j$, $l_i < l_j$, ale $d_i > d_j$.

Zamieniamy miejscami z_i i z_j . Po zamianie:

- z_i jest ukończone w chwili $l_j \leq d_j < d_i$, więc z_i jest nadal terminowe.
- z_j jest ukończone w chwili $l_i < l_j \leq d_j$, więc z_j jest nadal terminowe.

Po pewnej liczbie takich zamian otrzymujemy harmonogram spełniający warunki lematu. $\square$